

AGENTIC AI ENGINEER @ GOOGLE

A Personalized, Consolidated Roadmap

Synthesized from 3 uploaded roadmaps + roadmap.sh/coders.sh + live Google job-posting research

Prepared: August 16, 2026

0. What Went Into This — and What Didn't

Three uploaded roadmaps were fully read and mined: the @aiwithapoorvaa AI Engineer Track A/B roadmap, the "SYNTAX ERROR" 3-Month AI/ML roadmap, and the dhruvtechbytes 2026 Product-Based-Company (PBC) Prep Roadmap. I also pulled current data from roadmap.sh's AI Engineer and AI Agents roadmaps, the coders.sh AI Engineer roadmap, and — most important for calibrating this to Google specifically — live Google Careers postings for "Agentic AI" software engineering roles (searched today).

Could not be accessed: the four file:///C:/Users/... links point to files on your own computer, not the internet — I have no way to open local paths on your machine, only files you actually upload into this chat. Same for the "AI Engineering Pathways" Google Doc — no shareable link was given, and "Shivamani999/AI-Engineer" did not resolve to a public repo I could locate. If any of those four HTML roadmaps have content you specifically want folded in, upload them directly (or paste the text) and I'll merge them in a revision. Everything below is built from what I could verify.

1. Reality Check — What Google Actually Hires For

This matters more than any single roadmap. I searched live Google Careers listings for "Agentic AI" engineering roles (Agentic AI Infrastructure, AI Application & Agentic Development, Agentic AI/Trust & Safety, Google Cloud Agentic AI, Agentic Infrastructure — GBO). The pattern is consistent and important:

What the postings actually say	Implication for you
Every single one is titled 'Software Engineer' first, 'Agentic AI' second.	There is no separate, easier hiring track for 'AI roles' at Google. You go through the same coding + system design + Googleyness loop as any SWE.
Core language requirements are C++, Python, Java, Go, Kotlin — general-purpose, not 'just Python + LangChain'.	Track A-style roadmaps (LLM-app-only) are necessary but not sufficient. Strong fundamentals still matter.
Recurring phrases: 'multi-agent systems', 'tool-use / function calling', 'LLM orchestration (LangChain/LangGraph)', 'context engineering', 'grounding', 'vector databases (Vertex AI, Pinecone, Weaviate)'.	This is exactly the RAG + Agents + MCP layer covered by Track A (A8-A10) — your strongest existing resource for this part.
'Experience with large-scale infrastructure, distributed systems', 'ultra-low-latency backend', '7 years coding' for senior roles.	Backend depth (Track B: B3, B7, B8) is not optional filler for Google — it's a real differentiator, especially for infra-flavored agentic roles.
No posting mentions LeetCode, but Google's standard interview loop (unchanged for over a decade) is Coding/DSA rounds + System Design + Googleyness — confirmed by every current Google interview-prep source.	DSA is not optional. See Section 2.

2. Do You Need DSA? Yes — Verdict and Why

Two of your three source roadmaps (Track A/B and SYNTAX ERROR) barely mention data structures & algorithms — they're written for people targeting startup / freelance / applied-AI-engineer roles where DSA-heavy interviews are less common. The third, the PBC roadmap, is built by someone actively targeting product-based companies in 2026 and puts DSA (Striver's A2Z Sheet + daily LeetCode) as a full parallel pillar alongside System Design.

For a Google target specifically, the PBC roadmap is right and the other two are incomplete on this point. Google's interview loop for SWE roles — including every ‘Agentic AI’ posting reviewed above, since they're all leveled as SWE — still runs candidates through 2-4 coding/DSA rounds plus system design plus Googleyness. Skipping DSA because you're ‘an AI engineer, not a SWE’ will not get you past Google's phone screen, no matter how good your RAG pipeline is.

- Keep it: Striver's A2Z Sheet (pattern-wise) + 3–4 LeetCode problems/day once the sheet is done — exactly as laid out in the PBC roadmap.
- Focus areas that show up most in Google loops: Arrays/Strings, Binary Search, Trees & Graphs (BFS/DFS heavy), Dynamic Programming, Heaps, and Greedy — plus the ability to talk through time/space complexity out loud.
- Treat DSA as a parallel, ongoing track from month 1, not a phase you do “after” LLM/agent work — see the timeline in Section 5.

3. Gap Analysis — What Each Roadmap Gives You (and Doesn't)

Roadmap	Strongest for	Missing for a Google Agentic AI target
Track A/B — @aiwithapoorvaa	The actual agentic-engineering stack: prompting, tool calling, RAG, evaluation (Promptfoo), LLM security/prompt injection, MCP, local models. This is your best single source for the “agentic” half of the job.	No DSA. No Google-scale system design (only light backend). No Google-specific interview mechanics.
SYNTAX ERROR — 3-Month AI/ML	Classical ML/DL fundamentals: regression, CNNs, RNNs, backprop — useful for the “explain bias-variance / how does attention work” style of question and for Track-B depth.	Nothing on agents, RAG, MCP, LLM security, or production LLM engineering — the actual day-to-day of the role. No DSA, no system design.
PBC 2026 — dhruvtechbytes	Exactly the two things Google-specific candidates most often under-prepare: DSA (Striver A2Z) and HLD/LLD System Design, plus real interviewer-facing tactics (back-of-envelope estimation, concurrency, UML).	The “Agentic AI” phase here is a single Udemy course — thin compared to Track A's depth. No RAG detail, no MCP, no LLM security/evaluation.

The fix is simple: none of the three is complete alone, but together they cover the whole bar almost perfectly. The plan below merges them — PBC's DSA + System Design skeleton, Track A's agentic/RAG/MCP/security depth, and SYNTAX ERROR trimmed down to interview-relevant ML fundamentals only (you do not need the full 3-month deep-learning track — Track B already told you PyTorch-from-scratch is optional unless a specific role demands it).

4. Timeline — Realistic, Not Aspirational

Assumes 15–20 hrs/week, starting from working Python basics (per your Track A prerequisites). Three tracks run in parallel from the start rather than sequentially — this is the single biggest change from following any one source roadmap on its own.

Track	Duration	Cadence
Track A — Foundations → Agentic Engineering (Python/backend → LLM apps → RAG → Agents → MCP)	Months 1–6	Primary focus, sequential within itself
DSA (Striver A2Z → daily LeetCode)	Months 1–10, ongoing after	Parallel, every week, no skipping
System Design (HLD + LLD)	Months 5–9	Starts once backend + one real project exist to design around
ML/DL fundamentals (trimmed, interview-relevant only)	Months 2–4, light touch	Woven around Track A, not a separate 3-month block
Portfolio, Google-specific prep, mock interviews, applying	Months 8–12	Ramps up as projects complete

Total: roughly 10–12 months to a genuinely Google-ready application — not the 5–6 months Track A alone estimates (that number excludes DSA and system design entirely) and not the 3 months SYNTAX ERROR estimates (that roadmap doesn't include agentic engineering at all).

5. Month-by-Month Plan

Months 1–2 — Foundations, in parallel

- Python: OOP, error handling, file I/O, type hints, venv/uv, JSON, pytest (Track A, section A1).
- Git + Linux/CLI basics (Track A, A2).
- Start DSA immediately: Python refresher → Arrays/Strings/Binary Search on Striver's A2Z Sheet (PBC roadmap). Don't wait for “backend” to be done first.
- Minimum ML/math literacy: vectors/embeddings, cosine similarity intuition, 3Blue1Brown's “But what is a GPT?”, basic probability (Track A, A4). Skip the SYNTAX ERROR roadmap's calculus/eigenvalue-heavy week — not needed for this role and Track A explicitly flags it as unnecessary depth.

Checkpoint: a class-based Python script with tests, comfortable Git workflow, and Striver A2Z sheet started (Arrays + Binary Search patterns underway).

Months 2–4 — Backend + Core LLM Engineering

- FastAPI + SQL basics, build a small CRUD API (Track A, A3).
- LLM application engineering: system prompts, structured outputs, function/tool calling, streaming, context-window management (Track A, A5) — use Anthropic's official prompt-engineering tutorial, the single best resource across all sources reviewed.

- Lightweight eval habit: Promptfoo, 15-case test sets, before/after comparisons (Track A, A6). This is a Google-interview-friendly skill — “how do you know your system works” is a real question you'll get asked.
- LLM security: prompt injection (direct vs. indirect), OWASP Top 10 for LLM Apps, guardrails (Track A, A7) — directly relevant, since Google's “Agentic AI, Trust and Safety” postings explicitly call this out.
- DSA continues: Trees & Graphs (BFS/DFS), start Dynamic Programming basics.
- Trimmed ML fundamentals in the background: StatQuest playlist for the classic supervised/unsupervised concepts (SYNTAX ERROR roadmap), enough to answer conceptual interview questions — skip building CNNs/RNNs from scratch unless you later target a more ML-research-flavored team.

Checkpoint: working FastAPI service, a chatbot + structured extractor + one tool-calling assistant, an eval suite proving a prompt change helped, and one documented prompt-injection test against your own app.

Months 4–6 — RAG, Agents, MCP (the core of the role)

- RAG: parsing (including messy/scanned docs), chunking, embeddings, vector DBs, hybrid search, reranking, citations (Track A, A8). Google's job postings explicitly mention Vertex AI / Pinecone / Weaviate — make sure your project uses a real vector DB, not just an in-memory list.
- Agents: the agent loop, workflow vs. autonomous agent distinction, retries/guardrails (Track A, A9). Google postings repeatedly say “multi-agent systems” and “complex tool-use patterns” — build at least one project with 2+ coordinating agents, not just a single-agent demo.
- MCP: client/server/tool/resource model, build one real MCP server (Track A, A10).
- Local/open-weight models with Ollama — useful talking point for cost/latency tradeoff questions in system design rounds (Track A, A11).
- DSA continues: finish DP, Greedy, Heaps; start weekly virtual contests (PBC roadmap).

Checkpoint: a RAG app with cited answers over a real (including messy) document set, a multi-agent workflow project, one working MCP server — these become your two flagship portfolio projects.

Months 5–9 — System Design (parallel, starts once you have real projects to design around)

- HLD: functional/non-functional requirements gathering, back-of-envelope estimation first, scaling and cross-region discussion (PBC roadmap's Shrayansh Jain + TechPrep playlists).
- LLD: UML first, SOLID principles, concurrency and race-condition handling, design patterns (PBC roadmap's Prateek Narang course + Shrayansh Jain LLD playlist).
- Deliberately design at Google scale, not laptop scale: redo your RAG app and agent workflow as “handle 10,000 concurrent users” system-design exercises (Track B, B7) — this is exactly the kind of prompt Google system-design rounds use.
- Deploy at least one project for real: Docker + a live URL, environment variables handled properly (Track A, A12).

Checkpoint: you can whiteboard both your RAG system and your multi-agent system at production scale — estimations, bottlenecks, failure modes — not just describe how you built the demo.

Months 8–12 — Portfolio, Google-specific prep, applying

- Polish your 2 strongest projects (RAG app + multi-agent/MCP app), real GitHub commit history, clear READMEs (Track A, A13).
- Write 1–2 debugging case studies in Situation→Problem→Design→Breakage→Fix→Outcome format — Track A's format, genuinely useful for behavioral rounds too.

- DSA: shift to daily LeetCode (3–4/day), mixed patterns, timed mock rounds.
- Google-specific: read up on “Googleyness & Leadership” — Google's behavioral dimension alongside coding and system design — and prepare 5–6 STAR-format stories from your own project work (ambiguity handled, conflict, failure + recovery, initiative).
- Apply through Google Careers directly for roles matching the postings researched here: search “Agentic AI”, “AI Application and Agentic Development”, “Agentic Infrastructure” — and pursue a referral if you have any personal network at Google, since it materially changes response rates.
- Run timed mock interviews (peers, Pramp-style practice, or structured self-practice) for both coding and system design before applying — don't let real interviews be your first timed practice.

Checkpoint: application-ready — 2 strong, deployed, Google-scale-discussable projects; DSA at consistent daily-solve level; 5+ behavioral stories ready; resume tailored to the specific Google posting's language (‘multi-agent’, ‘tool-use’, ‘grounding’, ‘context engineering’ — mirror their vocabulary).

6. Consolidated Resource Map

Every resource below is pulled from one of your three uploaded roadmaps, roadmap.sh/coders.sh, or Google's own docs. Source is tagged so you know where to go back for full context.

Foundations

Topic	Resource	Source
Python OOP/errors/files/pytest	Real Python (OOP), Corey Schafer YouTube series, freeCodeCamp Pytest Tutorial	Track A — A1
Git & Linux/CLI	GitHub Docs Git Cheat Sheet, freeCodeCamp Git & GitHub Crash Course	Track A — A2
Linear algebra / embeddings intuition	Towards AI — Linear Algebra for ML, 3Blue1Brown — “But what is a GPT?”	Track A — A4
Classical ML concepts (for interview talking points only)	StatQuest ML playlist (Josh Starmer)	SYNTAX ERROR roadmap

Backend

Topic	Resource	Source
FastAPI + SQL	freeCodeCamp FastAPI Course, FastAPI official docs, freeCodeCamp SQL Full Course	Track A — A3
Deep backend (later, optional)	TestDriven.io JWT auth guide, Better Stack FastAPI+JWT+SQLAlchemy guide	Track A — B3

LLM & Agentic Engineering (core of the role)

Topic	Resource	Source
Prompting & tool calling	Anthropic Prompt Engineering Overview + Interactive Tutorial (GitHub), Anthropic — Effective Context Engineering for Agents	Track A — A5
Evaluation habit	Promptfoo docs/quickstart, DataCamp Promptfoo Tutorial	Track A — A6
LLM security	OWASP Top 10 for LLM Applications, Promptfoo OWASP red-teaming guide	Track A — A7
RAG	LangChain “RAG From Scratch”, Krish Naik RAG Crash Course, LangChain RAG docs, BuildRAG — Parsing PDF Documents	Track A — A8
Agents	aibuilderclub “AI Agents in 2026” guide, OpenAI Agents tool-calling tutorial	Track A — A9
MCP	Anthropic — Intro to MCP course, freeCodeCamp MCP servers with Python, Microsoft MCP for Beginners	Track A — A10
Local models	freeCodeCamp — Local AI Development with Ollama	Track A — A11
Agentic AI end-to-end bootcamp (LangChain + LangGraph)	Udemy — Complete Agentic AI Bootcamp with LangGraph and LangChain	PBC roadmap

Topic	Resource	Source
Current landscape check	roadmap.sh/ai-agents, roadmap.sh/ai-engineer, coders.sh AI Engineer roadmap	Web research (2026)

DSA

Topic	Resource	Source
Structured DSA sheet	Striver's A2Z Sheet — takeuforward.org/dsa/strivers-a2z-sheet-learn-dsa-a-to-z	PBC roadmap
Daily practice	LeetCode, pattern-wise, 3–4/day after the sheet; weekly/virtual contests	PBC roadmap
Interview execution habits	Brute force → discuss TC/SC → optimize → dry run before submitting	PBC roadmap

System Design (HLD + LLD)

Topic	Resource	Source
HLD	Shrayansh Jain HLD playlist, TechPrep interview-centric HLD playlist	PBC roadmap
LLD	Prateek Narang — Low Level System Design (Udemy), Shrayansh Jain LLD playlist	PBC roadmap
Google-scale practice	Redo Track A's RAG/agent builds at “10,000 users” scale	Track A — B7
Full cloud/deploy depth (optional, later)	freeCodeCamp — Docker & Kubernetes Full Course	Track A — B8

Deploy & Portfolio

Topic	Resource	Source
Docker + deployment	freeCodeCamp Docker Tutorial, freeCodeCamp Docker Handbook	Track A — A12
Portfolio & README	freeCodeCamp — How to Write a Good README	Track A — A13
Interview prep (general + SQL)	freeCodeCamp — How to Prepare for a Technical Interview, SQL Interview Questions Solved	Track A — A13

7. Google-Specific Prep Google's Own Postings Point To

Pulled directly from the live postings reviewed in Section 1 — things worth deliberately building toward, beyond the generic roadmap:

- Show multi-agent coordination, not just single-agent demos — “designing multi-agent systems” is called out explicitly.
- Use a real vector database in at least one project (Vertex AI Search, Pinecone, or Weaviate), not an in-memory FAISS toy — named explicitly in one posting.
- Be able to talk about grounding and context-engineering optimization, not just “prompting” — Anthropic's context-engineering article (already in A5) is directly relevant here.
- If you can, pick up model quantization / serving basics (vLLM, TGI, or Ollama's own quantized formats) — mentioned as a plus in more than one posting and covered lightly by Track A's Ollama section.
- General-purpose language depth still matters — Google's postings list C++, Java, Go, Kotlin alongside Python. You don't need to master all of them, but being able to write clean, idiomatic code in at least one beyond Python strengthens a Google application.
- Prepare Googleyness/behavioral stories around ambiguity, cross-team collaboration, and technical trade-off communication — phrased almost identically across every posting reviewed.

8. Weekly Time Budget (example, 18 hrs/week)

Track	Hours/week	Notes
Core Track A curriculum (current phase)	8–10 hrs	Sequential, follow Section 5
DSA	5–6 hrs	Never skip, even in “light” weeks
System Design (from Month 5 on)	2–3 hrs	Replaces some Track A hours once RAG/Agents are done
Portfolio/writing/review	1–2 hrs	Ramps up heavily in Months 8–12

9. Final Checklist Before Applying

1. Two deployed, documented flagship projects: one RAG system with citations over real (incl. messy) documents, one multi-agent/MCP workflow.
2. Both projects can be discussed at “10,000 users” system-design scale, not just “works on my laptop.”
3. Striver's A2Z Sheet complete; comfortable solving unseen medium LeetCode problems in ~30–40 min with a clear brute-force→optimize narration.
4. At least one HLD and one LLD mock interview completed with feedback.
5. 5–6 STAR-format Googleyness stories ready, pulled from real debugging/collaboration moments in your own projects.
6. Resume mirrors the vocabulary from actual Google postings (multi-agent, tool-use, grounding, context engineering, orchestration).

7. Applied directly on Google Careers to roles matching the titles researched in Section 1, and pursued a referral if any personal network exists.

Good luck — revisit Section 4/5 monthly and adjust pace honestly rather than rushing phases.